

Notes and Exercises from *Computability, Complexity, and Languages* by Martin D. Davis et al.

Anthony Ozog

Chapter 2: Programs and Computable Functions

Section 1. A Programming Languages

Notes

The first programming language that is defined is pretty simple. It only has the following parts

$V \leftarrow V + 1$	Increment	
$V \leftarrow V - 1$	Decrement*	1
IF $V \neq 0$ GOTO A	Conditional Branch	

* If $V = 0$ then keep $V = 0$.

Actually there are no negative numbers, they only kind of numbers the program works for are just in $\mathbb{N}$. Also for convention they call the input variables $X_1, X_2, X_3, \dots$ and the output variables $Y_1, Y_2, Y_3, \dots$ and local variables $Z_1, Z_2, Z_3, \dots$ often dropping the subscript.

X	Input	
Y	Output	2
Z	Local	

Also instructions are optionally “labelled” using the following notation.

$$A_1, B_1, C_1, D_1, E_1, A_2, B_2, C_2, D_2, E_2, A_3, \dots \quad 3$$

Once again often with the subscript dropped. E is often the label of the “exit” instruction, meaning it has no instructions and halts the program. Also instructions must be finite. Additionally local and output variables are initialized to 0. Finally this programming language is designated $\mathcal{L}$.

Section 2. Some Examples of Programs

Notes

Abbreviating a “pseudo”-instruction or set of instructions is called a *Macro* and the following is an example of a common one.

GOTO [L] 4

Where the IF part of the conditional branch is omitted. Another common macro that is seen a lot is

$V \leftarrow V'$ 5

where the V' is replaced by some other variable during the *macro expansion*. Here the book works hard to show how this assignment/initialization takes place and what the expansion looks like. As a mathematician it seems pretty simple that I can initialize a variable to whatever I want it to be, including outputs of functions on variables denoted $+$, $-$, $\times$, $\dots$, but it is cool to see how these abstractions occur in the language $\mathcal{L}$.

Exercises

1. Write a program in $\mathcal{L}$ that computes the function $f(x) = 3x$.

Solution

$$\begin{array}{l} Y \leftarrow X \\ Z \leftarrow 2 \\ \left. \begin{array}{l} \text{IF } Y \neq 0 \text{ GOTO } A \\ \text{GOTO } E \end{array} \right] \text{ Unnecessary} \\ [A] \quad Y \leftarrow X + Y \\ \quad Z \leftarrow Z - 1 \\ \quad \text{IF } Z \neq 0 \text{ GOTO } A \end{array}$$
 6

2. Write a program in $\mathcal{L}$ that solves Exercise 1 using no macros.

I am not gonna do this one since you can just fill in the macros.

3. Let $f(x) = 1$ if x is even; $f(x) = 0$ if x is odd. Write a program in $\mathcal{L}$ that computes f .

$$\begin{array}{l} [C] \quad \text{IF } X \neq 0 \text{ GOTO } A \\ \quad \text{GOTO } E \\ [A] \quad X \leftarrow X - 1 \\ \quad Y = Y + 1 \\ \quad \text{IF } X \neq 0 \text{ GOTO } B \\ \quad \text{GOTO } E \\ [B] \quad X \leftarrow X - 1 \\ \quad Y = Y - 1 \\ \quad \text{IF } X \neq 0 \text{ GOTO } A \end{array}$$
 7

4. Let $f(x) = 1$ if x is even; $f(x)$ undefined if x is odd. Write a program in $\mathcal{L}$ that computes f .

```

      Y ← 1
[C]   IF X ≠ 0 GOTO A
      GOTO E
[A]   X ← X − 1
      IF X ≠ 0 GOTO B
      GOTO A
[B]   X ← X − 1
      IF X ≠ 0 GOTO A

```

8

Section 3. Syntax

Notes

This section is a bit of repetition as well as a very detailed account of the language $\mathcal{L}$, so I will have brief notes.

The following are called *statements*

$$\begin{aligned} V &\leftarrow V + 1 \\ V &\leftarrow V - 1 \\ V &\leftarrow V \\ \text{IF } V \neq 0 \text{ GOTO } L \end{aligned} \quad 9$$

where V may be any variable and L may be any label. An *instruction* is either a statement or a label-statement pair. A *program* is a list (finite sequence) of instructions. The length of the list is the *length* of a program. The *empty program* is the program of length 0.

The *state of a program* $\mathcal{P}$ is a list of equations of the form $V = m$, where V is a variable and m is a number, including an equation for each variable that occurs in $\mathcal{P}$ and including no two equations with the same variable.

Let σ be a state of $\mathcal{P}$ and let V be a variable that occurs in σ . The *value of V at σ* is then the number q such that the equation $V = q$ is one of the equations making up σ . The *snapshot* or *instantaneous description* of a program $\mathcal{P}$ of length n is the pair (i, σ) where i is the instruction about to be executed and $1 \leq i \leq n + 1$ and σ is a state of $\mathcal{P}$. $i = n + 1$ is the “stop” instruction.

Page 27 defines all the cases of a *successor* of a snapshot.

A *computation* of a program $\mathcal{P}$ is a list

$$s_1, s_2, s_3, \dots, s_k \quad 10$$

s_i snapshots of $\mathcal{P}$, such that s_{i+1} is the successor of s_i for $i = 1, 2, 3, \dots, k - 1$ and s_k is *terminal*, meaning $i = n + 1$.

Exercises

1. Give a program $\mathcal{P}$ that such that for every computation $s_1, s_2, \dots, s_k$ of $\mathcal{P}$, $k = 5$.

Solution

$$\begin{aligned} Z &\leftarrow 0 \\ Z &\leftarrow Z + 1 \\ Z &\leftarrow Z + 1 \\ Z &\leftarrow Z + 1 \end{aligned} \quad 11$$

Section 4. Computable Functions

Notes

The *Initial State* of a program $\mathcal{P}$ in the language $\mathcal{L}$ for $r_1, r_2, \dots, r_m$ given numbers is the state σ of $\mathcal{P}$ where

$$X_1 = r_1, X_2 = r_2, \dots, X_m = r_m, Y = 0 \quad 12$$

and $V = 0$ for all other variables in $\mathcal{P}$. The *Initial Snapshot* is denoted $(1, \sigma)$.

Case 1. There exists a computation $s_1, s_2, \dots, s_k$ of $\mathcal{P}$ beginning with the initial snapshot. Then $\psi_{\mathcal{P}}^{(m)}(r_1, r_2, \dots, r_m)$ is the value of Y at the terminal snapshot s_k .

Case 2. There does not exist a computation. This means there is an infinite sequence $s_1, s_2, \dots$ beginning with the initial snapshot. Also, $\psi_{\mathcal{P}}^{(m)}(r_1, r_2, \dots, r_m)$ is undefined here.

A function g is *partially computable* if for some $r_1, r_2, \dots, r_m$,

$$g(r_1, r_2, \dots, r_m) = \psi_{\mathcal{P}}^{(m)}(r_1, r_2, \dots, r_m). \quad 13$$

Also when $g(r_1, r_2, \dots, r_m)$ is undefined so is $\psi_{\mathcal{P}}^{(m)}(r_1, r_2, \dots, r_m)$, and when g is defined they are equal. A function g is *total* if it is defined for all input $r_1, r_2, \dots, r_m$.

A function is *computable* if it is both partially computable and total

Exercises

- Let $\mathcal{P}$ be a program

$$\begin{array}{ll} & \text{IF } X \neq 0 \text{ GOTO } A \\ [A] & X \leftarrow X + 1 \\ & \text{IF } X \neq 0 \text{ GOTO } A \\ [A] & Y \leftarrow Y + 1 \end{array} \quad 14$$

What is $\psi_{\mathcal{P}}^{(1)}$?

Solution

It is the nowhere defined function.

- The same as Exercise 1 for the program

$$\begin{array}{ll} [B] & \text{IF } X \neq 0 \text{ GOTO } A \\ & Z \leftarrow Z + 1 \\ & \text{IF } Z \neq 0 \text{ GOTO } B \\ [A] & X \leftarrow X \end{array} \quad 15$$

Solution

$$\psi_{\mathcal{P}}^{(1)} = \begin{cases} 0 & \text{undefined} \\ x \neq 0 & 0 \end{cases} \quad 16$$

- The same as Exercise 1 for the empty program

Solution

$$\psi_{\mathcal{P}}^{(1)} = 0 \quad 17$$

4. Let $\mathcal{P}$ be the program

$$\begin{array}{ll}
 & Y \leftarrow X_1 \\
 [A] & \text{IF } X_2 \neq 0 \text{ GOTO } E \\
 & Y \leftarrow Y + 1 \\
 & Y \leftarrow Y + 1 \\
 & X_2 \leftarrow X_2 - 1 \\
 & \text{GOTO } A
 \end{array}
 \tag{18}$$

What is $\psi_{\mathcal{P}}^{(1)}(r_1)$? $\psi_{\mathcal{P}}^{(2)}(r_1, r_2)$? $\psi_{\mathcal{P}}^{(3)}(r_1, r_2, r_3)$?

Solution

$$\begin{aligned}
 \psi_{\mathcal{P}}^{(1)}(r_1) &= \text{undefined} \\
 \psi_{\mathcal{P}}^{(2)}(r_1, r_2) &= \begin{cases} (r_1, 0) & \text{undefined} \\ (r_1, r_2 \neq 0) & r_1 \end{cases} \\
 \psi_{\mathcal{P}}^{(3)}(r_1, r_2, r_3) &= \begin{cases} (r_1, 0, r_3) & \text{undefined} \\ (r_1, r_2 \neq 0, r_3) & r_1 \end{cases}
 \end{aligned}
 \tag{19}$$

5. Show that for every partially computable function $f(x_1, \dots, x_n)$ there is a number $m \geq 0$ such that f is computed by infinitely many programs of length m .

Proof

Let $f(x_1, \dots, x_n)$ be some partially computable function such that

$$f(x_1, \dots, x_n) = \psi_{\mathcal{P}^*}^{(n)}(x_1, \dots, x_n) \tag{20}$$

where $\mathcal{P}^*$ is a program of length l that computes f . Now take $m = l + 1$ and look at the set of all programs S of length m that begin with $\mathcal{P}^*$ and whose final instruction does not change the value of Y (the output) and does not contain a GOTO. It is easy to see that all programs in S compute f since the final instruction does not change Y and $\mathcal{P}^*$ would be terminal at that point. It is also easy to see that S is infinite since the final instruction could be any of the following

$$\begin{array}{l}
 Z_1 \leftarrow 0 \\
 Z_2 \leftarrow 0 \\
 Z_3 \leftarrow 0 \\
 \vdots \\
 Z_1 \leftarrow 1 \\
 Z_2 \leftarrow 1 \\
 Z_3 \leftarrow 1 \\
 \vdots \\
 Z_1 \leftarrow 2 \\
 \vdots
 \end{array}
 \tag{21}$$

■

6. (a) For every number $k \geq 0$, let f_k be the constant function $f_k(x) = k$. Show that for every k , f_k is computable.

Proof

Let $k \geq 0$ and consider the function $f_k(x) = k$. It is easy to see that f_k is total since it is equal to k for all x . Now all that is left to show is that f_k is partially computable. Considered the following program $\mathcal{P}$

$$Y \leftarrow k \qquad 22$$

where $\psi_{\mathcal{P}}^{(1)}(x) = k$ thus $\psi_{\mathcal{P}}^{(1)}(x) = f_k$. This means f_k is partially computable and since it is total f_k is computable for any k .



Section 5. More about Macros

Notes

Observe the following macro.

$$W \leftarrow f(V_1, \dots, V_n) \quad 23$$

where f is a partially computable function. This macro is the abbreviation of the following expansion.

$$\begin{aligned} & Z_m \leftarrow 0 \\ & Z_{m+1} \leftarrow V_1 \\ & Z_{m+2} \leftarrow V_2 \\ & \vdots \\ & Z_{m+n} \leftarrow V_n \\ & Z_{m+n+1} \leftarrow 0 \\ & Z_{m+n+2} \leftarrow 0 \\ & \vdots \\ & Z_{m+n+k} \leftarrow 0 \\ & \mathcal{Q}_m \\ [E_m] \quad & W \leftarrow Z_n \end{aligned} \quad 24$$

where m is large enough that these variables are never used in the main program. This next macro makes IF statement look more familiar.

$$\text{IF } P(V_1, \dots, V_n) \text{ GOTO } L \quad 25$$

where $P(V_1, \dots, V_n)$ is a computable predicate that evaluates to either

$$\text{TRUE} = 1, \quad \text{FALSE} = 0 \quad 26$$

which means Equation 25 is the expansion of

$$\begin{aligned} & Z \leftarrow P(V_1, \dots, V_n) \\ & \text{IF } Z \neq 0 \text{ GOTO } L \end{aligned} \quad 27$$

Note how all of these *macros* are just abbreviations of the original language $\mathcal{L}$ which means if anything is computable with these macros, or any new ones we define using $\mathcal{L}$, then it is computable in $\mathcal{L}$.

Exercises

1. Let $f(x), g(x)$ be computable functions and let $h(x) = f(g(x))$. Show that h is computable.

Proof Since f and g are computable there exists

$$\begin{aligned} f(x) &= \psi_{\mathcal{P}_f}^{(1)}(x) \\ &\text{and} \\ g(x) &= \psi_{\mathcal{P}_g}^{(1)}(x) \end{aligned} \quad 28$$

for programs $\mathcal{P}_f$ and $\mathcal{P}_g$ respectively. Then h can be computed using the following program

$$\begin{aligned} X &\leftarrow x \\ Z &\leftarrow \psi_{\mathcal{P}_g}^{(1)}(X) \\ Y &\leftarrow \psi_{\mathcal{P}_f}^{(1)}(Z) \end{aligned} \tag{29}$$

which is easily see to compute h .

2. Show by constructing a program that the predicate $x_1 \leq x_2$ is computable.

Consider the following program

$$\begin{aligned} &Z_1 \leftarrow x_1 \\ &Z_2 \leftarrow x_2 \\ &\text{IF } Z_2 \neq 0 \text{ GOTO } P \\ [P] &\text{IF } Z_1 \neq 0 \text{ GOTO } S \\ [T] &Y \leftarrow 1 \\ &\text{GOTO } E \\ [F] &Y \leftarrow 0 \\ &\text{GOTO } E \\ [S] &Z_1 \leftarrow Z_1 - 1 \\ &Z_2 \leftarrow Z_2 - 1 \\ &\text{IF } Z_1 \neq 0 \text{ GOTO } C \\ &\text{GOTO } T \\ [C] &\text{IF } Z_2 \neq 0 \text{ GOTO } S \\ &\text{GOTO } F \end{aligned} \tag{30}$$

Chapter 3: Primitive Recursive Functions

Section 1. Composition

Notes

Definition. Let f be a function of k variables and let $g_1, \dots, g_k$ be functions of n variables. Let

$$h(x_1, \dots, x_n) = f(g_1(x_1, \dots, x_n), \dots, g_k(x_1, \dots, x_n)). \quad 31$$

Then h is said to be obtained from f and $g_1, \dots, g_k$ by *composition*.

Theorem 1.1. If h is obtained from the (partially) computable functions $f, g_1, \dots, g_k$ by composition, then h is (partially) computable.

Section 2. Recursion

Notes

Definition Let k be a fixed number and

$$\begin{aligned} h(0) &= k \\ h(t+1) &= g(t, h(t)) \end{aligned} \quad 32$$

where g is some given *total* function of two variables. Then h is obtained from g by *recursion*.

Theorem 2.1. Let h be obtained from g as in Equation 32, and let g be computable. Then h is computable.

Theorem 2.2. Let h be obtained from f and g in the following way,

$$\begin{aligned} h(x_1, \dots, x_n, 0) &= f(x_1, \dots, x_n) \\ h(x_1, \dots, x_n, t+1) &= g(t, h(x_1, \dots, x_n, t), x_1, \dots, x_n) \end{aligned} \quad 33$$

and let f and g be computable. Then h is also computable.

Section 3. PRC Classes

Notes

Consider the following functions

$$\begin{aligned} s(x) &= x + 1 \\ n(x) &= 0 \end{aligned} \quad 34$$

and the projection functions

$$u_i^n(x_1, \dots, x_n) = x_i, \quad 1 \leq i \leq n. \quad 35$$

The functions s , n , and u_i^n are called *initial functions*.

Definition. A class of total functions $\mathcal{C}$ is called a Primitive Recursively Closed (PRC) class if

1. the initial functions belong to $\mathcal{C}$,
2. a function obtained from functions belonging to $\mathcal{C}$ by either composition or recursion also belongs to $\mathcal{C}$.

Theorem 3.1. The class of computable functions is a PRC class.

Proof. Just show that the *initial functions* are computable.

Definition A function is called *primitive recursive* if it can be obtained from the initial function by a finite number of applications of composition and recursion.

Theorem 3.3. A function is primitive recursive if and only if it belong to every PRC class.

Corollary 3.4 Every primitive recursive function is computable.

To note, primitive recursion functions are only a subset of all computable functions. Later in the book they are going to construct a function that is not primitive recursive but is computable.

Exercises

1. Let $\mathcal{C}$ be a PRC class, and let g_1, g_2, g_3, g_4 belong to $\mathcal{C}$. Show that if

$$\begin{aligned} h_1(x, y, z) &= g(z, y, x) \\ h_2(x) &= g(x, x, x) \\ h_3(w, x, y, z) &= h_1(g_3(x, x, x), z, g_4(2, g_4(y, z))) \end{aligned} \quad 36$$

then h_1, h_2, h_3 also belong to $\mathcal{C}$.

Proof.

Since h_1, h_2, h_3 are all obtained from g_1, g_2, g_3, g_4 and g_1, g_2, g_3, g_4 belong to $\mathcal{C}$ and $\mathcal{C}$ is a PRC that means h_1, h_2, h_3 belong to $\mathcal{C}$ by the definition.

2. Show that the class of all total functions is a PRC class.

Proof.

Property 1 Let $\mathcal{C}_t$ be the class of total functions. And since

$$\begin{aligned} s(x) &= x + 1 \\ n(x) &= 0 \\ u_i^n(x_1, \dots, x_n) &= x_i, \end{aligned} \quad 37$$

the initial functions, are defined everywhere they are in $\mathcal{C}_t$.

Property 2 let f and g be total functions and let h be some function obtained from either composition or recursion of f and g . Then when ever f and g are defined so is h and since f and g are total so is h , then h belongs to $\mathcal{C}_t$. Thus, $\mathcal{C}_t$ is a PRC class.

Section 4. Some Primitive Recursive Functions

Notes

Remember, all primitive recursive functions are computable. Now, consider the following example.

$$f(x, y) = x + y \quad 38$$

Then we can define f recursively as follows.

$$\begin{aligned} f(x, 0) &= x, \\ f(x, y + 1) &= f(x, y) + 1, \end{aligned} \quad 39$$

which can be rewritten as

$$\begin{aligned} f(x, 0) &= u_1^1(x), \\ f(x, y + 1) &= g(y, f(x, y), x), \end{aligned} \quad 40$$

where $g(x_1, x_2, x_3) = s(u_2^3(x_1, x_2, x_3))$ and remember that u and s are initial functions. From this point the book gives a few other examples but I get the picture.

Exercises

1. Give a detailed arguments that x^y , $p(x)$, and $x \div y$ are primitive recursive. I am only going to do one of these.

$$f(x, y) = x^y$$

Recursively the function looks like the following,

$$\begin{aligned} x^0 &= 1, \\ x^{y+1} &= x^y \cdot x. \end{aligned} \quad 41$$

The base case can be expressed as,

$$f(x, 0) = s(n(x)). \quad 42$$

Then for the recursive case expressed it as

$$f(x, y + 1) = g(x, f(x, y), y) \quad 43$$

where $g(x_1, x_2, x_3) = h(u_2^3(x_1, x_2, x_3), u_1^3(x_1, x_2, x_3))$ and that $h(x, y) = x \cdot y$ and was show to be primitive recursive in the book on page 45.

2. Show that for each k , the function $f(x) = k$ is primitive recursive.

The function f can be defined recursively as the following,

$$\begin{aligned} f(0) &= k, \\ f(x + 1) &= f(x) \end{aligned} \quad 44$$

using the initial functions this can be expressed as

$$\begin{aligned} f(0) &= \underbrace{s(s(s(\dots s(n(x))\dots)))}_{k \text{ times}} \\ f(x + 1) &= g(f(n(x))) \end{aligned} \quad 45$$

where $g(x_1) = u_1^1(x_1)$.

Section 5. Primitive Recursive Predicates